

UNIT 5

LearnSmart –Content Analysis, Classification & Retrieval Engine(Part 1 of 2)

CO Mapped: CO5

Research Based Problems Mapped: 1, 2,3, & 4.

Type: Practical + Analytical Activity (Project Part 1 of 2)

Problem Statement

E-learners across the globe struggle to find suitable content while learning technical courses online. A learner at the beginner level may be overwhelmed by advanced material, while an advanced learner may waste time on content that is too simple. There is no intelligent system that analyses, classifies, and retrieves content based on the learner's level.

Problems faced without this system:

- E-learning platforms cannot automatically label content difficulty — it is done manually and inconsistently.
- Keyword search returns irrelevant results without ranking by relevance or learner level.
- No retrieval model exists to compare probabilistic vs vector-space approaches for educational content.

Your engine must solve these by building:

- A preprocessing and semantic analysis pipeline (TF-IDF + PCA).
- A VSM-based retrieval system and a Probabilistic (BM25) retrieval system.
- A 5-classifier comparison framework (ID3, SVM, Quantum SVM, MLP, ANN) for content difficulty labeling.

Objective

Develop the analysis and classification engine of LearnSmart that:

- Preprocesses e-learning content descriptions using NLP techniques.
- Applies Semantic Analysis (TF-IDF) and reduces dimensionality using PCA.
- Implements a VSM-based retrieval system and a Probabilistic (BM25) retrieval system and compares them.
- Trains and compares 5 classifiers (ID3, SVM, Quantum SVM, MLP, ANN) for content difficulty classification.
- Saves all outputs (features, retrieval indexes, classifications) as structured files for Activity 10.

Dataset

Sample Dataset

Students must use the following sample dataset to test and verify their implementations before switching to a real dataset.

doc_id	content_title	level	description	difficulty
C001	Introduction to Python Programming	Beginner	Variables, loops, functions – basic Python syntax	Easy
C002	Data Structures and Algorithms	Intermediate	Arrays, trees, sorting, graph algorithms	Moderate
C003	Machine Learning with TensorFlow	Advanced	Neural networks, backpropagation, deep learning	Tough
C004	Web Development using HTML & CSS	Beginner	HTML tags, CSS styling, responsive design basics	Easy
C005	Database Management Systems	Intermediate	SQL, normalization, ER diagrams, transactions	Moderate
C006	Operating Systems Concepts	Advanced	Process scheduling, memory management, deadlocks	Tough
C007	Software Engineering Principles	Intermediate	SDLC, UML, agile methodology, testing strategies	Moderate
C008	Computer Networks Fundamentals	Beginner	OSI model, IP addressing, basic protocols	Easy

Real Dataset

- Coursera Courses Dataset (Kaggle), edX Course Dataset (Kaggle)
- Use at least 100 records. Fields required: doc_id, content_title, description, difficulty_label (Easy / Moderate / Tough).

Tasks to be Performed

Task 1: Text Preprocessing, Semantic Analysis & PCA

1A – Text Preprocessing

- Load the dataset (CSV/JSON).
- Apply the following NLP preprocessing steps:
 - Lowercasing all text

- Tokenization (split into words)
- Stopword removal (using NLTK stopwords)
- Lemmatization (using WordNetLemmatizer)
- Display a sample of 3 preprocessed documents.

1B – Semantic Analysis (TF-IDF)

- Apply TF-IDF vectorization on the preprocessed corpus using sklearn TfidfVectorizer.
- Display the TF-IDF matrix shape and a sample of the top 10 terms for 3 documents.
- Save the TF-IDF matrix as tfidf_matrix.json.

1C – PCA Dimensionality Reduction

- Apply PCA on the TF-IDF feature matrix.
- Determine the optimal number of components by plotting the explained variance ratio (scree plot – print values if plot unavailable).
- Retain components that explain at least 90% of the variance.
- Print the number of components selected and total variance retained.
- Save PCA-transformed features as pca_features.json.

Task 2: VSM-Based Retrieval System

- Build a Vector Space Model (VSM) retrieval system using the TF-IDF vectors from Task 1.
- Accept a learner query (e.g., 'machine learning neural network') as input.
- Compute cosine similarity between the query vector and all document vectors.
- Return and display the Top 5 most relevant content items with their similarity scores and difficulty labels.
- Save the VSM index as vsm_index.json.

Expected Output Format

Rank | doc_id | content_title | difficulty | similarity_score

Task 3: Probabilistic Retrieval System (BM25) & Comparison

3A – BM25 Implementation

- Implement the BM25 (Best Match 25) probabilistic retrieval model using the rank_bm25 Python library.
- Use the same learner query from Task 2.
- Return and display the Top 5 results with BM25 scores and difficulty labels.
- Save the BM25 index as bm25_index.json.

3B – VSM vs BM25 Comparison

- Create a side-by-side comparison table of the top-5 results from VSM (Task 2) vs BM25 (Task 3) for the same query.
- Discuss: Which model returns more relevant results? Why might they differ?
- Identify at least 2 advantages and 2 disadvantages of each model.

Task 4: Content Difficulty Classification (5 Classifiers)

Train all 5 classifiers on the PCA-transformed features (pca_features.json) with difficulty labels (Easy / Moderate / Tough). Use an 80/20 train-test split.

4A – ID3 (Decision Tree with Entropy)

- Train: `sklearn.tree.DecisionTreeClassifier(criterion='entropy')`
- Evaluate: Accuracy, Precision, Recall, F1-Score (weighted).

4B – SVM (Support Vector Machine)

- Train: `sklearn.svm.SVC(kernel='rbf', C=1.0, gamma='scale')`
- Evaluate: Accuracy, Precision, Recall, F1-Score (weighted).

4C – Quantum SVM (QSVM)

- Simulate using Qiskit Machine Learning: `qiskit_machine_learning.algorithms.QSVC`
- Use ZZFeatureMap quantum kernel (on reduced 2-4 PCA components for simulation efficiency).
- Evaluate: Accuracy, Precision, Recall, F1-Score (weighted).

4D – MLP (Multi-Layer Perceptron)

- Train: `sklearn.neural_network.MLPClassifier(hidden_layer_sizes=(100,50), max_iter=500)`
- Evaluate: Accuracy, Precision, Recall, F1-Score (weighted).

4E – ANN (Artificial Neural Network)

- Build a Keras/TensorFlow Sequential model:
 - Input layer → `Dense(128, relu)` → `Dense(64, relu)` → `Dense(3, softmax)`
 - Compile: `optimizer='adam', loss='sparse_categorical_crossentropy'`
 - Train for 50 epochs with validation split=0.2
- Evaluate: Accuracy, Precision, Recall, F1-Score (weighted).

4F – Classifier Comparison & Best Model

- Create a comparison table with columns: Classifier | Accuracy | Precision | Recall | F1-Score.
- Identify and justify the best classifier based on overall F1-Score.
- Save all predictions as `classified_content.json` (`doc_id`, `content_title`, `predicted_difficulty`, `true_difficulty`).

Task 5: Analysis & Interpretation

5A – PCA Analysis

- How many PCA components were selected and what percentage of variance do they retain?
- How did dimensionality reduction affect the classifiers' performance compared to full TF-IDF features?

5B – Retrieval Analysis

- Compare VSM vs BM25 results: which model returned more relevant content for your test query?
- When would you prefer BM25 over VSM in a real e-learning platform?

5C – Classifier Analysis

- Which classifier performed best on this dataset and why?
- Compare ID3 vs ANN: interpretability vs accuracy trade-off.
- What are the practical limitations of using Quantum SVM in a real system?

5D – Short Report (300–400 words)

- Summarise your findings from Tasks 1–4.
- Discuss how the classified content and retrieval indexes will be used in Activity 10.

What Students Must Submit

- Python code for all 5 tasks (single .py file or Jupyter notebook)
- Output files: tfidf_matrix.json, pca_features.json, vsm_index.json, bm25_index.json, classified_content.json
- Classifier comparison table (screenshot or printed output)
- VSM vs BM25 comparison table (screenshot or printed output)
- Short report – Analysis & Interpretation (300–400 words)

Expected Outcome

Students should successfully:

- Build a complete NLP preprocessing and semantic analysis pipeline with PCA.
- Implement and compare VSM and BM25 retrieval systems.
- Train all 5 classifiers and identify the best performer for difficulty classification.
- Produce all structured output files required as inputs for Activity 10.

Evaluation Rubric

Task	Excellent (4)	Good (3)	Satisfactory (2)	Poor (1)	No Attempt (0)
Preprocessing, Semantic Analysis & PCA	All preprocessing steps correct; TF-IDF matrix accurate; PCA applied with explained variance plotted and optimal components justified.	Minor issues in PCA or TF-IDF computation.	Basic preprocessing and TF-IDF done; PCA attempted.	Incomplete or incorrect preprocessing / PCA.	Not attempted.
VSM-Based Retrieval System	Correct cosine similarity computation; top-5 results returned with meaningful scores; query processing accurate.	Minor issues in similarity computation or result ranking.	Basic VSM implemented with partial results.	Incorrect VSM implementation.	Not attempted.
Probabilistic Retrieval (BM25) & Comparison	BM25 correctly implemented; top-5 results returned; clear comparison table of VSM vs BM25 with analysis.	Minor BM25 parameter issues; basic comparison done.	Basic BM25 attempted; limited comparison.	Incorrect BM25 or no comparison.	Not attempted.
Classifier Implementation & Comparison (ID3, SVM, QSVM, MLP, ANN)	All 5 classifiers trained correctly; comparison table with Accuracy, Precision, Recall, F1; best classifier identified with justification.	4 of 5 classifiers correct; comparison mostly complete.	3 classifiers implemented; basic comparison.	Fewer than 3 classifiers or incorrect evaluation.	Not attempted.
Analysis & Interpretation	Clear, insightful report; discusses PCA impact, retrieval comparison, classifier performance; all output files saved correctly.	Good analysis with minor gaps; most files saved.	Basic analysis; some files missing.	Poor or incomplete analysis.	Not attempted.